Lab 2 – Równoległe sito Eratostenesa w MPI (mpi4py)

Antoni Smółka

Uruchomienie

Struktura: `src/` – programy, `ares/` – skrypty SLURM i ich wyniki, `raport/` – sprawozdanie.

Uruchomienie na Aresie (z katalogu `lab2`):

```
chmod a+x src/*.py
sbatch ares/run_test_sm.sbatch
sbatch ares/run_sieve.sbatch 1000000000
squeue -u $USER
```

Interaktywnie (krótkie testy):

```
srun --overlap -A plgar2026-cpu -N 1 --tasks-per-node=4 -p plgrid -t 20:00 --pty /bin/bash
module load scipy-bundle/2021.10-intel-2021b
export SLURM_OVERLAP=1
mpiexec src/sieve_mpi.py 100 --print
```

1. Quickstart i `test_sm.py`

`try.py` wypisuje rangę, liczbę procesów i nazwę węzła dla każdego procesu. `test_sm.py`: proces 0 alokuje okno 25 liczb `int`, pozostałe procesy alokują 0 bajtów i przez `Shared_query(0)` dostają wskaźnik na pamięć procesu 0. Każdy proces wpisuje swój numer w swoje 5 komórek, `Fence` synchronizuje zapisy, po czym proces 0 widzi całą tablicę:

```
Zawartość współdzielonej tablicy:
[0 0 0 0 0 1 1 1 1 1 2 2 2 2 2 3 3 3 3 3 4 4 4 4 4]
```

2. Część podstawowa – `sieve_mpi.py`

Szukamy liczb pierwszych w $A = [2, n]$. Każda liczba złożona z A ma dzielnik pierwszy w $B = [2, \lfloor\sqrt{n}\rfloor]$.

1. **Podzielniki** B – każdy proces sam liczy klasycznym sitem (to tylko $\sqrt{n}$ liczb; replikacja obliczeń jest tańsza niż ich rozsyłanie).
2. **Dekompozycja domenowa** – $C = [\lfloor\sqrt{n}\rfloor + 1, n]$ dzielimy na p bloków; proces r dostaje $[\, lo + r \cdot |C|/p,\ lo + (r+1) \cdot |C|/p \,)$ (rozmiary różnią się co najwyżej o 1).
3. **Sito lokalne** – dla każdego podzielnika q proces wykreśla jego wielokrotności w swoim bloku, zaczynając od $\max(q^2, \lceil start/q \rceil \cdot q)$. Wykreślanie odbywa się *segmentami po 1 MB* (patrz niżej).
4. **Zebranie wyniku** – `gather` liczności, a następnie `Gatherv` tablic liczb pierwszych do procesu 0 (kolektywnie, bez pętli `send/recv`).

Optymalizacja – sito segmentowane. Pierwsza wersja wykreślała każdym podzielnikiem cały blok procesu naraz (`is_prime[first::q] = False`). Dla $n = 10^8$ to ~1,2 tys. przejść z krokiem q przez tablicę ~100 MB, które nie mieszczą się w cache. Sito było ograniczone przepustowością pamięci i przyspieszenie przy 8 procesach wynosiło tylko 2×. Po podzieleniu bloku na kawałki po 2^{20} liczb (wszystkie podzielniki przechodzą przez jeden kawałek, póki jest w cache) wersja jednoprocesowa przyspieszyła ~3× i poprawiło się skalowanie.

3. Część zaawansowana – `sieve_shared.py`

Proponowana modyfikacja: zamiast prywatnej tablicy w każdym procesie i przesyłania wyników, wszystkie procesy węzła operują na **jednej tablicy flag `is_prime[0..n]`** (1 bajt na liczbę), zaalokowanej przez proces 0 w oknie `MPI.Win.Allocate_shared`. Pozostałe procesy uzyskują do niej dostęp przez `Shared_query(0)`.

```
node = COMM_WORLD.Split_type(COMM_TYPE_SHARED)        # procesy z tego samego węzła
win  = Win.Allocate_shared(n+1 jeśli rank 0, inaczej 0, comm=node)
każdy proces: is_prime[mój blok C] = 1
Fence
proces 0: sito na [0, sqrt n] bezpośrednio w is_prime   # podzielniki liczone raz
Fence                                                   # podzielniki widoczne dla wszystkich
każdy proces: odczyt podzielników z is_prime[0..sqrt n],
              wykreślanie w is_prime[mój blok C]        # bloki rozłączne → brak wyścigów
Fence                                                   # tablica flag kompletna
każdy proces: lokalne liczby pierwsze z bloku, offset = Exscan(liczność)
win_p = Allocate_shared(suma liczności); primes[offset : offset+liczność] = lokalne
Fence                                                   # wynik w primes[], posortowany
```

Różnice względem wersji z komunikatami:

- **Podzielniki liczone raz** (przez proces 0) i *czytane* z pamięci przez resztę – bez replikacji obliczeń i bez rozsyłania.
- **Brak zbierania wyników komunikatami.** Tablica flag jest jedna; listę liczb pierwszych procesy budują równolegle w drugiej tablicy współdzielonej. Jedyne komunikaty to dwie liczby (`exscan`, `allreduce`) na proces, potrzebne do wyznaczenia offsetów.
- **Synchronizacja przez `Fence`** zamiast przez komunikaty. Wymagana jest tam, gdzie jeden proces czyta dane zapisane przez inny (podzielniki, gotowa tablica).
- **Poprawność:** każdy proces pisze tylko do swojego, rozłącznego bloku, więc w fazie wykreślania nie są potrzebne zamki.
- **Ograniczenie:** pamięć współdzielona działa tylko w obrębie jednego węzła. Program sprawdza to (`Split_type`) i przerywa się, gdy procesy są na różnych węzłach. Rozszerzenie na wiele węzłów: model hybrydowy, czyli okno współdzielone w każdym węźle plus `Gatherv` między węzłami.
- **Rozważony i odrzucony wariant:** podział *podzielników* zamiast przedziału (każdy proces wykreśla wielokrotności swoich q w całej tablicy). Zapisy byłyby wtedy bezpieczne logicznie, bo wszyscy wpisują tylko 0, ale obciążenie byłoby skrajnie nierówne (liczba wielokrotności to ok. n/q, a małe q dominują). Do tego wszystkie procesy pisałyby po całej tablicy (false sharing, brak lokalności cache).

4. Wyniki

Poprawność sprawdzona dla $n \in \{2, 10, 100, 10^6, 5\,000\,011, 10^8\}$ i $p \in \{1, 2, 3, 4, 7, 8\}$. Obie wersje dają ten sam wynik, zgodny z sitem referencyjnym, np. $\pi(10^6) = 78\,498$ i $\pi(10^8) = 5\,761\,455$.

Pomiary lokalne

MacBook, 4 rdzenie wydajne + 4 energooszczędne, MPICH 4, $n = 10^8$, czas w sekundach; *total* obejmuje zebranie wszystkich liczb pierwszych.

p	`sieve_mpi` total	przysp.	`sieve_shared` sito	`sieve_shared` total	przysp.
1	0,236	1,0	0,147	0,287	1,0
2	0,129	1,8	0,084	0,171	1,7
4	0,076	3,1	0,050	0,095	3,0
8	0,080	3,0	0,055	0,095	3,0

Wnioski z pomiarów lokalnych:

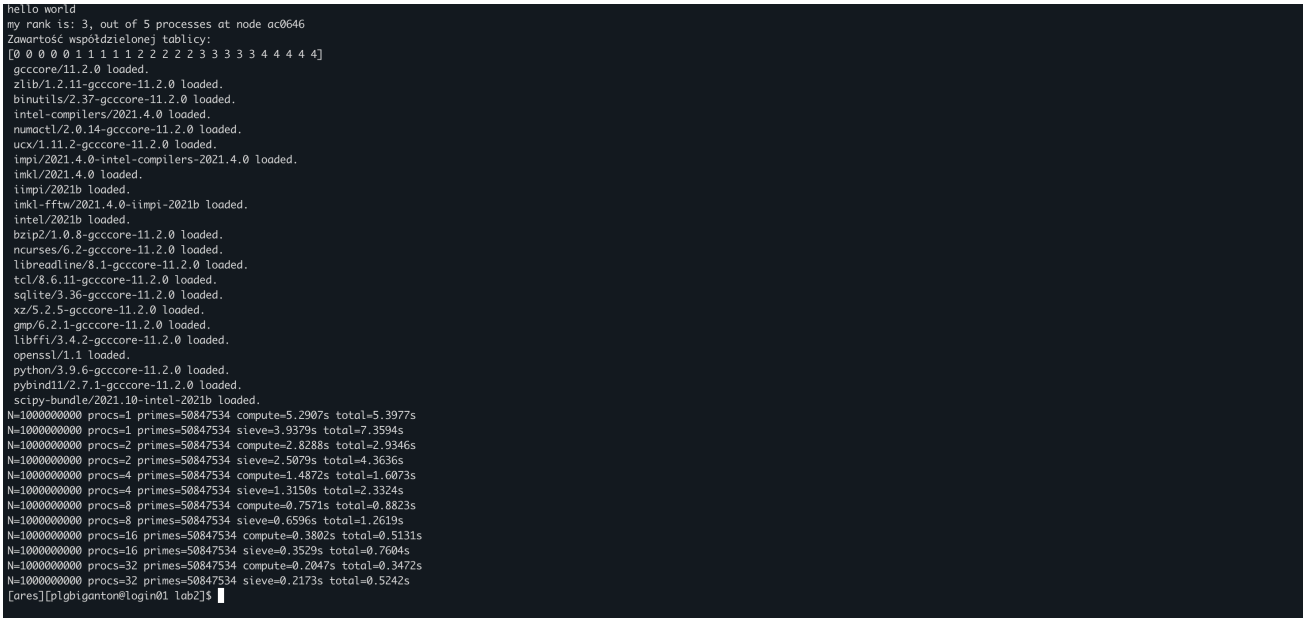
- Do 4 procesów przyspieszenie jest bliskie liniowemu. Powyżej 4 procesy trafiają na rdzenie energooszczędne i przyspieszenie przestaje rosnąć. Na Aresie (jednorodne rdzenie) skalowanie trwa dalej, do 32 procesów (niżej).
- Kolumna „sito” w wersji współdzielonej to samo wykreślanie. Całkowity czas obu wersji jest porównywalny, bo dużą część stanowi budowanie listy ~5,8 mln liczb pierwszych (`nonzero`). Pierwsza wersja, w której proces 0 sam przeglądał całą tablicę, prawie się nie skalowała (0,27 s → 0,20 s przy 8 procesach). Dopiero równoległe wpisywanie pod offsety z `Exscan` usunęło to wąskie gardło.
- Wersja współdzielona zużywa mniej pamięci: jedna tablica n bajtów na węzeł zamiast kopii wyników w procesie 0 po `Gatherv`.

Ares

Zadanie wsadowe `run_sieve.sbatch` (job 21492462): jeden węzeł, Intel MPI 2021.4, Python 3.9, $n = 10^9$. Obie wersje zwróciły $\pi(10^9) = 50\,847\,534$, co jest poprawną wartością. Quickstart i `test_sm.py` (job 21492461) dały na węźle ac0646 oczekiwany wynik (sekcja 1).

```
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
[ares][plgbiganton@login01 lab2]$   cat test_sm_*.out sieve_*.out
 gcccore/11.2.0 loaded.
 zlib/1.2.11-gcccore-11.2.0 loaded.
 binutils/2.37-gcccore-11.2.0 loaded.
 intel-compilers/2021.4.0 loaded.
 numactl/2.0.14-gcccore-11.2.0 loaded.
 ucx/1.11.2-gcccore-11.2.0 loaded.
 impi/2021.4.0-intel-compilers-2021.4.0 loaded.
 imkl/2021.4.0 loaded.
 iimpi/2021b loaded.
 imkl-fftw/2021.4.0-iimpi-2021b loaded.
 intel/2021b loaded.
 bzip2/1.0.8-gcccore-11.2.0 loaded.
 ncurses/6.2-gcccore-11.2.0 loaded.
 libreadline/8.1-gcccore-11.2.0 loaded.
 tcl/8.6.11-gcccore-11.2.0 loaded.
 sqlite/3.36-gcccore-11.2.0 loaded.
 xz/5.2.5-gcccore-11.2.0 loaded.
 gmp/6.2.1-gcccore-11.2.0 loaded.
 libffi/3.4.2-gcccore-11.2.0 loaded.
 openssl/1.1 loaded.
 python/3.9.6-gcccore-11.2.0 loaded.
 pybind11/2.7.1-gcccore-11.2.0 loaded.
 scipy-bundle/2021.10-intel-2021b loaded.
size=5, rank=0
hello world
my rank is: 0, out of 5 processes at node ac0646
size=5, rank=4
hello world
my rank is: 4, out of 5 processes at node ac0646
size=5, rank=2
hello world
my rank is: 2, out of 5 processes at node ac0646
size=5, rank=1
hello world
my rank is: 1, out of 5 processes at node ac0646
size=5, rank=3
hello world
my rank is: 3, out of 5 processes at node ac0646
Zawartość współdzielonej tablicy:
[0 0 0 0 0 1 1 1 1 1 2 2 2 2 2 3 3 3 3 3 4 4 4 4 4]
 gcccore/11.2.0 loaded.
 zlib/1.2.11-gcccore-11.2.0 loaded.
 binutils/2.37-gcccore-11.2.0 loaded.
```

Rysunek 1: Wynik `try.py` i `test_sm.py` na Aresie



Rysunek 2: Wynik `run_sieve.sbatch` na Aresie

p	`sieve_mpi` obliczenia	`sieve_mpi` total	przysp.	`sieve_shared` sito	`sieve_shared` total	przysp.
1	5,291	5,398	1,0	3,938	7,359	1,0
2	2,829	2,935	1,8	2,508	4,364	1,7
4	1,487	1,607	3,4	1,315	2,332	3,2
8	0,757	0,882	6,1	0,660	1,262	5,8
16	0,380	0,513	10,5	0,353	0,760	9,7
32	0,205	0,347	15,6	0,217	0,524	14,0

(czas w sekundach; przyspieszenie liczone dla czasu *total* względem $p = 1$ tej samej wersji)

Wnioski z Aresa:

- Część obliczeniowa skaluje się prawie liniowo. W `sieve_mpi` przyspieszenie obliczeń przy 32 procesach wynosi $5{,}29/0{,}205 \approx 26$ (efektywność ok. 80%), a wykreślanie w `sieve_shared` przyspiesza ok. 18×. Tak powinno być, bo procesy nie komunikują się w trakcie sita.
- Przyspieszenie całkowite jest niższe (15,6× i 14×, efektywność ok. 45–50%). Wynika to z części, która skaluje się słabo albo wcale: zebranie ~50,8 mln liczb pierwszych (≈400 MB jako `int64`) do jednego miejsca oraz stały koszt startu. W `sieve_mpi` sam `Gatherv` zajmuje ok. 0,1–0,14 s niezależnie od p, więc przy 32 procesach to już ~40% czasu (prawo Amdahla).
- Na Aresie wersja współdzielona jest całościowo wolniejsza niż wersja z komunikatami (0,52 s vs 0,35 s przy $p = 32$), choć samo wykreślanie ma podobny czas. Różnica powstaje w fazie budowania wyniku: alokacja drugiego okna 400 MB i zapisy do pamięci procesu 0. Prawdopodobna przyczyna (niezweryfikowana pomiarem): okno `Allocate_shared` z rozmiarem tylko w procesie 0 leży fizycznie na jednym węźle NUMA. Procesy z drugiego gniazda piszą wtedy do pamięci zdalnej, a Intel MPI realizuje `Gatherv` w obrębie węzła własnymi zoptymalizowanymi kopiami.
- Możliwa poprawka: każdy proces alokuje w oknie własny kawałek (`Allocate_shared` z niezerowym rozmiarem w każdym procesie). Pamięć jest wtedy lokalna dla procesu, a pozostali i tak widzą ją przez `Shared_query(r)`.
- Zaleta wersji współdzielonej pozostaje: proces 0 nie trzyma kopii wyników, a cała tablica flag (1 GB) istnieje w węźle raz.